Let I, J \subseteq [n]. If A is a n by n matrix over some ring, denote A[I, J] as the determinant of

https://gemini.google.com/app/635d6c3578862a05

User prompt: Let I, J \subseteq [n]. If A is a n by n matrix over some ring, denote A[I, J] as the determinant of the submatrix by selecting rows I and columns J. Prove in lean4 that det(A) A^{adj}[J^C, I^C] = det(A)^{n-k} A[I, J], where A^adj is the adjugate. Research mathlib apis, and focus on writing clean code.

Response: In general, Jacobi's complementary minor identity includes a permutation sign factor $\operatorname{sgn}(\sigma_{I,J}) = (-1)^{\sum_{i\in I} i + \sum_{j\in J} j}$ when $I \neq J$:

$$\det(A)\, A^{\mathrm{adj}}[J^c, I^c] = \operatorname{sgn}(\sigma_{I,J}) \det(A)^{n-k} A[I, J]$$

To see why the sign is necessary when $I \neq J$, consider $n = 2$, $k = 1$, and $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ with $A^{\mathrm{adj}} = \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Taking $I = \{1\}$ and $J = \{2\}$ gives $I^c = \{2\}$ and $J^c = \{1\}$, so $A[I, J] = b$ while $A^{\mathrm{adj}}[J^c, I^c] = (A^{\mathrm{adj}})_{1,2} = -b$, introducing a factor of -1. When $I = J$ (principal minors) or when the row/column partitions differ by an even permutation, $\operatorname{sgn}(\sigma_{I,J}) = 1$ and the unsigned identity holds on the nose.

Proof Strategy & Mathlib 4 APIs

Because `Matrix.det` in Mathlib is defined only on square matrices `Matrix m m R` where the row and column index types coincide, a $k \times k$ submatrix minor $A[I, J]$ and its $(n-k) \times (n-k)$ complementary minor $A^{\mathrm{adj}}[J^c, I^c]$ are most cleanly represented via index equivalences `eI eJ : ι₁ ⊕ ι₂ ≃ n` (where `ι₁` indexes the k chosen positions and `ι₂` indexes the $n-k$ complementary positions).

In block form relative to `eI` and `eJ`, the identity `A * A.adjugate = A.det • 1` implies:

$$\begin{pmatrix} A_{I,J} & A_{I,J^c} \\ A_{I^c,J} & A_{I^c,J^c} \end{pmatrix} \begin{pmatrix} I_k & (A^{\mathrm{adj}})_{J,I^c} \\ 0 & (A^{\mathrm{adj}})_{J^c,I^c} \end{pmatrix} = \begin{pmatrix} A_{I,J} & 0 \\ A_{I^c,J} & \det(A) I_{n-k} \end{pmatrix}$$

Taking determinants of both sides using Mathlib's block-triangular determinant lemmas yields the identity directly over any commutative ring R, without needing to invert $\det(A)$ or assume A is nonsingular.

Mathlib Module	Key Declarations Used	Purpose
`Mathlib.LinearAlgebra.Matrix.Adjugate`	`Matrix.adjugate`, `Matrix.mul_adjugate`	Rewrites $A \cdot A^{\mathrm{adj}} = \det(A) \cdot I_n$
`Mathlib.LinearAlgebra.Matrix.Block`	`Matrix.fromBlocks`, `Matrix.fromBlocks_multiply`, `Matrix.fromBlocks_inj`, `Matrix.fromBlocks_one`, `Matrix.fromBlocks_smul`	Block decomposition and block matrix multiplication
`Mathlib.LinearAlgebra.Matrix.Determinant.Basic`	`Matrix.det_fromBlocks_zero₂₁`, `Matrix.det_fromBlocks_zero₁₂`, `Matrix.det_submatrix_equiv_self`, `Matrix.det_permute`	Determinants of block-triangular and reindexed matrices
`Mathlib.LinearAlgebra.Matrix.Basic`	`Matrix.submatrix_mul_equiv`, `Matrix.submatrix_one_equiv`	Pulls matrix multiplication through reindexing equivalences

Lean 4 Formalization

The code below provides:

1. **det_submatrix_mul_adjugate_minor**: The core block-determinant identity `(A.submatrix eI eJ).det * A.adjugate[Jᶜ, Iᶜ] = A.det ^ |ι₂| * A[I, J]`.
2. **det_mul_adjugate_minor**: The general theorem for arbitrary partitions `eI eJ : ι₁ ⊕ ι₂ ≃ n` with the explicit permutation sign `Equiv.Perm.sign (eJ.symm.trans eI)`.
3. **det_mul_adjugate_principal_minor**: The unsigned identity $\det(A)A^{\mathrm{adj}}[I^c, I^c] = \det(A)^{n-k}A[I, I]$ for any `I : Finset n` with `I.card = k`.
4. **det_mul_adjugate_finset_minor**: The concrete subset formulation for `I J : Finset (Fin n)` of cardinality k using `Finset.orderIsoOfFin`.

```
import Mathlib.LinearAlgebra.Matrix.Adjugate
import Mathlib.LinearAlgebra.Matrix.Block
import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
import Mathlib.Data.Finset.Sort

open Matrix

variable {n m ι₁ ι₂ R : Type*}
  [Fintype n] [DecidableEq n]
  [Fintype m] [DecidableEq m]
  [Fintype ι₁] [DecidableEq ι₁]
  [Fintype ι₂] [DecidableEq ι₂]
  [CommRing R]

/-- Reindexing rows by `eI` and columns by `eJ` multiplies the determinant by
the sign of the relative permutation `eJ.symm.trans eI`. -/
lemma det_submatrix_equiv (A : Matrix n n R) (eI eJ : m ≃ n) :
    (A.submatrix eI eJ).det = Equiv.Perm.sign (eJ.symm.trans eI) • A.det := by
  have h : A.submatrix eI eJ = (A.submatrix (eJ.symm.trans eI) id).submatrix eJ eJ := by
    ext i j
    simp
  rw [h, det_submatrix_equiv_self, det_permute]

/-- **Core Block-Matrix Identity**:
For any row partition `eI : ι₁ ⊕ ι₂ ≃ n` and column partition `eJ : ι₁ ⊕ ι₂ ≃ n`,
`(A.submatrix eI eJ).det * A^{\text{adj}}[J^c, I^c] = A.det ^ |ι₂| * A[I, J]`. -/
theorem det_submatrix_mul_adjugate_minor (A : Matrix n n R) (eI eJ : ι₁ ⊕ ι₂ ≃ n) :
    (A.submatrix eI eJ).det *
      (A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)).det =
    A.det ^ Fintype.card ι₂ *
      (A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)).det := by
  let A₁₁ := A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)
  let A₁₂ := A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inr)
  let A₂₁ := A.submatrix (eI ∘ Sum.inr) (eJ ∘ Sum.inl)
  let A₂₂ := A.submatrix (eI ∘ Sum.inr) (eJ ∘ Sum.inr)
  let B₁₁ := A.adjugate.submatrix (eJ ∘ Sum.inl) (eI ∘ Sum.inl)
  let B₁₂ := A.adjugate.submatrix (eJ ∘ Sum.inl) (eI ∘ Sum.inr)
  let B₂₁ := A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inl)
  let B₂₂ := A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)
  have hA : A.submatrix eI eJ = fromBlocks A₁₁ A₁₂ A₂₁ A₂₂ := by
    ext (i | i) (j | j) <;> rfl
  have hB : A.adjugate.submatrix eJ eI = fromBlocks B₁₁ B₁₂ B₂₁ B₂₂ := by
```

```
    ext (i | i) (j | j) <;> rfl
  -- Expand `(A * A.adjugate).submatrix eI eI = A.det • 1` in block form
  have hAB : fromBlocks A₁₁ A₁₂ A₂₁ A₂₂ * fromBlocks B₁₁ B₁₂ B₂₁ B₂₂ =
      fromBlocks (A.det • 1) 0 0 (A.det • 1) := by
    rw [← hA, ← hB, submatrix_mul_equiv, mul_adjugate, submatrix_smul,
        submatrix_one_equiv, ← fromBlocks_one, fromBlocks_smul, smul_zero]
  rw [fromBlocks_multiply, fromBlocks_inj] at hAB
  obtain ⟨-, h₁₂, -, h₂₂⟩ := hAB
  -- Multiply the reindexed `A` by the block upper-triangular matrix `fromBlocks 1 B₁₂ 0 B₂₂`
  have h_mul : A.submatrix eI eJ * fromBlocks 1 B₁₂ 0 B₂₂ =
      fromBlocks A₁₁ 0 A₂₁ (A.det • 1) := by
    rw [hA, fromBlocks_multiply]
    simp [h₁₂, h₂₂]
  calc (A.submatrix eI eJ).det * B₂₂.det
    _ = (A.submatrix eI eJ * fromBlocks 1 B₁₂ 0 B₂₂).det := by
        rw [det_mul, det_fromBlocks_zero₂₁, det_one, one_mul]
    _ = (fromBlocks A₁₁ 0 A₂₁ (A.det • 1)).det := by
        rw [h_mul]
    _ = A.det ^ Fintype.card ι₂ * A₁₁.det := by
        rw [det_fromBlocks_zero₁₂, det_smul, det_one, mul_one, mul_comm]

/-- **Jacobi's Complementary Minor Identity (General Partition Form)**:
`det(A) * A^{\text{adj}}[J^c, I^c] = sgn(eJ⁻¹ ∘ eI) • (det(A)^{|I^c|} * A[I, J])`. -/
theorem det_mul_adjugate_minor (A : Matrix n n R) (eI eJ : ι₁ ⊕ ι₂ ≃ n) :
    A.det * (A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)).det =
    Equiv.Perm.sign (eJ.symm.trans eI) •
      (A.det ^ Fintype.card ι₂ * (A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)).det) := by
  have h := det_submatrix_mul_adjugate_minor A eI eJ
  rw [det_submatrix_equiv, smul_mul_assoc] at h
  rw [← h, smul_smul, Int.units_mul_self, one_smul]

/-- **Unsigned Principal Minor Identity (`I = J`)**:
For any subset `I : Finset n` of cardinality `k`,
`det(A) * A^{\text{adj}}[I^c, I^c] = det(A)^{n - k} * A[I, I]`. -/
theorem det_mul_adjugate_principal_minor (A : Matrix n n R) (I : Finset n)
    {k : ℕ} (hI : I.card = k) :
    A.det * (A.adjugate.submatrix (Subtype.val : Iᶜ → n) (Subtype.val : Iᶜ → n)).det =
    A.det ^ (Fintype.card n - k) *
      (A.submatrix (Subtype.val : I → n) (Subtype.val : I → n)).det := by
  -- Bridge `{x // x ∉ I}` and `↥(Iᶜ)` via `Finset.mem_compl`
  let eCompl : {x // x ∉ I} ≃ ↥(Iᶜ) :=
    { toFun := fun x => ⟨x.1, Finset.mem_compl.mpr x.2⟩
      invFun := fun x => ⟨x.1, Finset.mem_compl.mp x.2⟩
      left_inv := fun _ => rfl
      right_inv := fun _ => rfl }
  let eI : ↥I ⊕ ↥(Iᶜ) ≃ n :=
    (Equiv.sumCongr (Equiv.refl ↥I) eCompl.symm).trans (Equiv.sumCompl (· ∈ I))
  have h := det_submatrix_mul_adjugate_minor A eI eI
  have hcard : Fintype.card ↥(Iᶜ) = Fintype.card n - k := by
    rw [Fintype.card_coe, Finset.card_compl, hI]
  simpa [eI, eCompl, det_submatrix_equiv_self, hcard] using h
```

```
/-- Canonical equivalence `Fin k ⊕ Fin (n - k) ≃ Fin n` induced by `I : Finset (Fin n)`
of cardinality `k`, preserving the natural order on `I` and `Iᶜ`. -/
noncomputable def finsetSumComplEquiv {n k : ℕ} (I : Finset (Fin n)) (hI : I.card = k) :
    Fin k ⊕ Fin (n - k) ≃ Fin n :=
  have hIc : (Iᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hI]
  let e₁ : Fin k ≃ ↥I := (I.orderIsoOfFin hI).toEquiv
  let e₂ : Fin (n - k) ≃ {x // x ∉ I} :=
    (Iᶜ.orderIsoOfFin hIc).toEquiv.trans
      { toFun := fun x => ⟨x.1, Finset.mem_compl.mp x.2⟩
        invFun := fun x => ⟨x.1, Finset.mem_compl.mpr x.2⟩
        left_inv := fun _ => rfl
        right_inv := fun _ => rfl }
  (Equiv.sumCongr e₁ e₂).trans (Equiv.sumCompl (· ∈ I))

/-- **Jacobi's Complementary Minor Identity for `I, J ⊆ [n]`**:
Given `I, J : Finset (Fin n)` of size `k`, denotes `A[I, J]` and `A^{\text{adj}}[J^c, I^c]`
via their canonical order-preserving enumerations `orderIsoOfFin`. -/
theorem det_mul_adjugate_finset_minor {n k : ℕ} (A : Matrix (Fin n) (Fin n) R)
    (I J : Finset (Fin n)) (hI : I.card = k) (hJ : J.card = k)
    (hIc : (Iᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hI])
    (hJc : (Jᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hJ]) :
    A.det ∗
      (A.adjugate.submatrix
        (Subtype.val ∘ (Jᶜ.orderIsoOfFin hJc))
        (Subtype.val ∘ (Iᶜ.orderIsoOfFin hIc))).det =
    Equiv.Perm.sign ((finsetSumComplEquiv J hJ).symm.trans (finsetSumComplEquiv I hI)) •
      (A.det ^ (n - k) ∗
        (A.submatrix
          (Subtype.val ∘ (I.orderIsoOfFin hI))
          (Subtype.val ∘ (J.orderIsoOfFin hJ))).det) := by
  simpa [finsetSumComplEquiv, Function.comp_def] using
    det_mul_adjugate_minor A (finsetSumComplEquiv I hI) (finsetSumComplEquiv J hJ)
```

---

*User prompt: ✖ [1796/1796] Building DetPoly.Scratch (1.2s) warning: DetPoly/Scratch.lean: imports `Mathlib.LinearAlgebra.Matrix.Adjugate` from package `mathlib`, which is designed for use with the module system; consider adding `module` to the start of this file warning: DetPoly/Scratch.lean: imports `Mathlib.LinearAlgebra.Matrix.Block` from package `mathlib`, which is designed for use with the module system; consider adding `module` to the start of this file warning: DetPoly/Scratch.lean: imports `Mathlib.LinearAlgebra.Matrix.Determinant.Basic` from package `mathlib`, which is designed for use with the module system; consider adding `module` to the start of this file warning: DetPoly/Scratch.lean: imports `Mathlib.Data.Finset.Sort` from package `mathlib`, which is designed for use with the module system; consider adding `module` to the start of this file trace: .> LEAN_PATH=/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/MD4Lean/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/BibtexQuery/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/UnicodeBasic/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/Cli/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/leansqlite/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/batteries/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/Qq/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/aesop/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/proofwidgets/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/importGraph/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/LeanSearchClient/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/plausible/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/checkdecls/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/doc-gen4/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/mathlib/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/build/lib/lean /home/grant/.elan/toolchains/leanprover--lean4---v4.35.0/bin/lean /home/grant/proj/lean/Determinantal-Polynomials/DetPoly/Scratch.lean -o /home/grant/proj/lean/Determinantal-Polynomials/.lake/build/lib/lean/DetPoly/Scratch.olean -i /home/grant/proj/lean/Determinantal-Polynomials/.lake/build/lib/lean/DetPoly/Scratch.ilean -c /home/grant/proj/lean/Determinantal-Polynomials/.lake/build/ir/DetPoly/Scratch.c --setup /home/grant/proj/lean/Determinantal-Polynomials/.lake/build/ir/DetPoly/Scratch.setup.json --json warning: DetPoly/Scratch.lean:7:0: The module doc-string for a file should be the first command after the imports and any necessary `set_option` commands. Please, add a module doc-string (`/-! ... -/`) before `open Matrix`. Hint: Type `m(odule docstring) + [tab]` to insert a template via snippet. Note: This linter can be disabled with `set_option linter.style.header false` error: DetPoly/Scratch.lean:19:76: unsolved goals n : Type u_1 m : Type u_2 R : Type u_5 inst✝⁴ : Fintype n inst✝³ : DecidableEq n inst✝² : Fintype m inst✝¹ : DecidableEq m inst✝ : CommRing R A : Matrix n n R eI eJ : m ≃ n h : A.submatrix ⇑eI ⇑eJ = (A.submatrix (⇑(eJ.symm.trans eI)) id).submatrix ⇑eJ ⇑eJ ⊢ ↑↑(Equiv.Perm.sign (eJ.symm.trans eI)) * A.det = Equiv.Perm.sign (eJ.symm.trans eI) • A.det error: DetPoly/*

Scratch.lean:49:8: Tactic `rewrite` failed: Did not find an occurrence of the pattern submatrix 1 ⇑?e ⇑?e in the target expression (A.det • submatrix 1) ⇑eI ⇑eI = fromBlocks (A.det • 1) 0 0 (A.det • 1) n : Type u_1 ι₁ : Type u_3 ι₂ : Type u_4 R : Type u_5 inst✝⁶ : Fintype n inst✝⁵ : DecidableEq n inst✝⁴ : Fintype ι₁ inst✝³ : DecidableEq ι₁ inst✝² : Fintype ι₂ inst✝¹ : DecidableEq ι₂ inst✝ : CommRing R A : Matrix n n R eI eJ : ι₁ ⊕ ι₂ ≃ n A₁₁ : Matrix ι₁ ι₁ R := A.submatrix (⇑eI ∘ Sum.inl) (⇑eJ ∘ Sum.inl) A₁₂ : Matrix ι₁ ι₂ R := A.submatrix (⇑eI ∘ Sum.inl) (⇑eJ ∘ Sum.inr) A₂₁ : Matrix ι₂ ι₁ R := A.submatrix (⇑eI ∘ Sum.inr) (⇑eJ ∘ Sum.inl) A₂₂ : Matrix ι₂ ι₂ R := A.submatrix (⇑eI ∘ Sum.inr) (⇑eJ ∘ Sum.inr) B₁₁ : Matrix ι₁ ι₁ R := A.adjugate.submatrix (⇑eJ ∘ Sum.inl) (⇑eI ∘ Sum.inl) B₁₂ : Matrix ι₁ ι₂ R := A.adjugate.submatrix (⇑eJ ∘ Sum.inl) (⇑eI ∘ Sum.inr) B₂₁ : Matrix ι₂ ι₁ R := A.adjugate.submatrix (⇑eJ ∘ Sum.inr) (⇑eI ∘ Sum.inl) B₂₂ : Matrix ι₂ ι₂ R := A.adjugate.submatrix (⇑eJ ∘ Sum.inr) (⇑eI ∘ Sum.inr) hA : A.submatrix ⇑eI ⇑eJ = fromBlocks A₁₁ A₁₂ A₂₁ A₂₂ hB : A.adjugate.submatrix ⇑eJ ⇑eI = fromBlocks B₁₁ B₁₂ B₂₁ B₂₂ ⊢ (A.det • submatrix 1) ⇑eI ⇑eI = fromBlocks (A.det • 1) 0 0 (A.det • 1) error: DetPoly/Scratch.lean:80:49: failed to synthesize instance of type class Compl (Type u_1) Hint: Type class instance resolution failures can be inspected with the `set_option trace.Meta.synthInstance true` command. error: DetPoly/Scratch.lean:80:72: failed to synthesize instance of type class Compl (Type u_1) Hint: Type class instance resolution failures can be inspected with the `set_option trace.Meta.synthInstance true` command. Some required targets logged failures: - DetPoly.Scratch

Response: The four compiler errors and two warnings come from specific interactions in Lean `v4.35.0` / current Mathlib:

1. **Line 19 (`det_submatrix_equiv`)**: `Matrix.det_permute` produces `↑↑(Equiv.Perm.sign σ) * A.det` (multiplication via coercion $\mathbb{Z}^\times \to \mathbb{Z} \to R$) rather than `Equiv.Perm.sign σ • A.det`. Adding `Units.smul_def, zsmul_eq_mul` bridges • and *.

2. **Line 49 (`det_submatrix_mul_adjugate_minor`)**: `Matrix.submatrix_smul` in Mathlib is pointfree in the reindexing maps (`submatrix (r • M) = r • submatrix M`), which turns `submatrix (A.det • 1) eI eI` into `(A.det • submatrix 1) eI eI` and blocks `submatrix_one_equiv` from matching. Closing hAB with `ext (i | i) (j | j) <;> simp [one_apply]` avoids that fragile rewrite chain altogether.

3. **Line 80 (`det_mul_adjugate_principal_minor`)**: In `(Subtype.val : Iᶜ → n)`, Lean sees that the left side of → must be a `Type`, so it coerces `I` to `↥I : Type _`'s subtype *before* applying `ᶜ` and looks for `Compl (Type u_1)`. Writing `↥(Iᶜ)` (as was done later in the proof) forces `Iᶜ` to be computed as a `Finset n` first.

4. **Module & Header Warnings**: Lean `v4.35.0` uses the `module` system across Mathlib, and `linter.style.header` expects a `/-! ... -/` module docstring immediately after imports.

Here is the corrected, warning-free code for `DetPoly/Scratch.lean`:

```
module

public import Mathlib.LinearAlgebra.Matrix.Adjugate
public import Mathlib.LinearAlgebra.Matrix.Block
public import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
public import Mathlib.Data.Finset.Sort

/-!
# Jacobi's Complementary Minor Identity

This file proves that for an `n × n` matrix `A` over a commutative ring `R`,
`det(A) * A^{\text{adj}}[J^c, I^c] = \text{sgn}(\sigma_{I,J}) • (det(A)^{n-k} * A[I, J])`.
-/

public section

open Matrix

variable {n m ι₁ ι₂ R : Type*}
  [Fintype n] [DecidableEq n]
  [Fintype m] [DecidableEq m]
  [Fintype ι₁] [DecidableEq ι₁]
  [Fintype ι₂] [DecidableEq ι₂]
  [CommRing R]

/-- Reindexing rows by `eI` and columns by `eJ` multiplies the determinant by
the sign of the relative permutation `eJ.symm.trans eI`. -/
```

```
lemma det_submatrix_equiv (A : Matrix n n R) (eI eJ : m ≃ n) :
    (A.submatrix eI eJ).det = Equiv.Perm.sign (eJ.symm.trans eI) • A.det := by
  have h : A.submatrix eI eJ = (A.submatrix (eJ.symm.trans eI) id).submatrix eJ eJ := by
    ext i j
    simp
  rw [h, det_submatrix_equiv_self, det_permute, Units.smul_def, zsmul_eq_mul]

/-- **Core Block-Matrix Identity**:
For any row partition `eI : ι₁ ⊕ ι₂ ≃ n` and column partition `eJ : ι₁ ⊕ ι₂ ≃ n`,
`(A.submatrix eI eJ).det * A^{\text{adj}}[J^c, I^c] = A.det ^ |ι₂| * A[I, J]`. -/
theorem det_submatrix_mul_adjugate_minor (A : Matrix n n R) (eI eJ : ι₁ ⊕ ι₂ ≃ n) :
    (A.submatrix eI eJ).det *
      (A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)).det =
    A.det ^ Fintype.card ι₂ *
      (A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)).det := by
  let A₁₁ := A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)
  let A₁₂ := A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inr)
  let A₂₁ := A.submatrix (eI ∘ Sum.inr) (eJ ∘ Sum.inl)
  let A₂₂ := A.submatrix (eI ∘ Sum.inr) (eJ ∘ Sum.inr)
  let B₁₁ := A.adjugate.submatrix (eJ ∘ Sum.inl) (eI ∘ Sum.inl)
  let B₁₂ := A.adjugate.submatrix (eJ ∘ Sum.inl) (eI ∘ Sum.inr)
  let B₂₁ := A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inl)
  let B₂₂ := A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)
  have hA : A.submatrix eI eJ = fromBlocks A₁₁ A₁₂ A₂₁ A₂₂ := by
    ext (i | i) (j | j) <;> rfl
  have hB : A.adjugate.submatrix eJ eI = fromBlocks B₁₁ B₁₂ B₂₁ B₂₂ := by
    ext (i | i) (j | j) <;> rfl
  -- Expand `(A * A.adjugate).submatrix eI eI = A.det • 1` in block form
  have hAB : fromBlocks A₁₁ A₁₂ A₂₁ A₂₂ * fromBlocks B₁₁ B₁₂ B₂₁ B₂₂ =
      fromBlocks (A.det • 1) 0 0 (A.det • 1) := by
    rw [← hA, ← hB, submatrix_mul_equiv, mul_adjugate]
    ext (i | i) (j | j) <;> simp [one_apply]
  rw [fromBlocks_multiply, fromBlocks_inj] at hAB
  obtain ⟨-, h₁₂, -, h₂₂⟩ := hAB
  -- Multiply the reindexed `A` by the block upper-triangular matrix `fromBlocks 1 B₁₂ 0 B₂₂`
  have h_mul : A.submatrix eI eJ * fromBlocks 1 B₁₂ 0 B₂₂ =
      fromBlocks A₁₁ 0 A₂₁ (A.det • 1) := by
    rw [hA, fromBlocks_multiply]
    simp [h₁₂, h₂₂]
  calc (A.submatrix eI eJ).det * B₂₂.det
    _ = (A.submatrix eI eJ * fromBlocks 1 B₁₂ 0 B₂₂).det := by
        rw [det_mul, det_fromBlocks_zero₂₁, det_one, one_mul]
    _ = (fromBlocks A₁₁ 0 A₂₁ (A.det • 1)).det := by
        rw [h_mul]
    _ = A.det ^ Fintype.card ι₂ * A₁₁.det := by
        rw [det_fromBlocks_zero₁₂, det_smul, det_one, mul_one, mul_comm]

/-- **Jacobi's Complementary Minor Identity (General Partition Form)**:
`det(A) * A^{\text{adj}}[J^c, I^c] = sgn(eJ⁻¹ ∘ eI) • (det(A)^{|I^c|} * A[I, J])`. -/
theorem det_mul_adjugate_minor (A : Matrix n n R) (eI eJ : ι₁ ⊕ ι₂ ≃ n) :
    A.det * (A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)).det =
    Equiv.Perm.sign (eJ.symm.trans eI) •
```

```
      (A.det ^ Fintype.card ι₂ * (A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)).det) := by
  have h := det_submatrix_mul_adjugate_minor A eI eJ
  rw [det_submatrix_equiv, smul_mul_assoc] at h
  rw [← h, smul_smul, Int.units_mul_self, one_smul]

/-- **Unsigned Principal Minor Identity (`I = J`)**:
For any subset `I : Finset n` of cardinality `k`,
`det(A) * A^{\text{adj}}[I^c, I^c] = det(A)^{n - k} * A[I, I]`. -/
theorem det_mul_adjugate_principal_minor (A : Matrix n n R) (I : Finset n)
    {k : ℕ} (hI : I.card = k) :
    A.det * (A.adjugate.submatrix (Subtype.val : ↥(Iᶜ) → n) (Subtype.val : ↥(Iᶜ) → n)).det =
    A.det ^ (Fintype.card n - k) *
      (A.submatrix (Subtype.val : ↥I → n) (Subtype.val : ↥I → n)).det := by
  let eCompl : {x // x ∉ I} ≃ ↥(Iᶜ) :=
    { toFun := fun x => ⟨x.1, Finset.mem_compl.mpr x.2⟩
      invFun := fun x => ⟨x.1, Finset.mem_compl.mp x.2⟩
      left_inv := fun _ => rfl
      right_inv := fun _ => rfl }
  let eI : ↥I ⊕ ↥(Iᶜ) ≃ n :=
    (Equiv.sumCongr (Equiv.refl ↥I) eCompl.symm).trans (Equiv.sumCompl (· ∈ I))
  have h := det_submatrix_mul_adjugate_minor A eI eI
  have hcard : Fintype.card ↥(Iᶜ) = Fintype.card n - k := by
    rw [Fintype.card_coe, Finset.card_compl, hI]
  simpa [eI, eCompl, det_submatrix_equiv_self, hcard] using h

/-- Canonical equivalence `Fin k ⊕ Fin (n - k) ≃ Fin n` induced by `I : Finset (Fin n)`
of cardinality `k`, preserving the natural order on `I` and `Iᶜ`. -/
noncomputable def finsetSumComplEquiv {n k : ℕ} (I : Finset (Fin n)) (hI : I.card = k) :
    Fin k ⊕ Fin (n - k) ≃ Fin n :=
  have hIc : (Iᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hI]
  let e₁ : Fin k ≃ ↥I := (I.orderIsoOfFin hI).toEquiv
  let e₂ : Fin (n - k) ≃ {x // x ∉ I} :=
    (Iᶜ.orderIsoOfFin hIc).toEquiv.trans
      { toFun := fun x => ⟨x.1, Finset.mem_compl.mp x.2⟩
        invFun := fun x => ⟨x.1, Finset.mem_compl.mpr x.2⟩
        left_inv := fun _ => rfl
        right_inv := fun _ => rfl }
  (Equiv.sumCongr e₁ e₂).trans (Equiv.sumCompl (· ∈ I))

/-- **Jacobi's Complementary Minor Identity for `I, J ⊆ [n]`**:
Given `I, J : Finset (Fin n)` of size `k`, denotes `A[I, J]` and `A^{\text{adj}}[J^c, I^c]`
via their canonical order-preserving enumerations `orderIsoOfFin`. -/
theorem det_mul_adjugate_finset_minor {n k : ℕ} (A : Matrix (Fin n) (Fin n) R)
    (I J : Finset (Fin n)) (hI : I.card = k) (hJ : J.card = k)
    (hIc : (Iᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hI])
    (hJc : (Jᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hJ]) :
    A.det *
      (A.adjugate.submatrix
        (Subtype.val ∘ (Jᶜ.orderIsoOfFin hJc))
        (Subtype.val ∘ (Iᶜ.orderIsoOfFin hIc))).det =
    Equiv.Perm.sign ((finsetSumComplEquiv J hJ).symm.trans (finsetSumComplEquiv I hI)) •
      (A.det ^ (n - k) *
```

```
      (A.submatrix
        (Subtype.val ∘ (I.orderIsoOfFin hI))
        (Subtype.val ∘ (J.orderIsoOfFin hJ))).det) := by
  simpa [finsetSumComplEquiv, Function.comp_def] using
    det_mul_adjugate_minor A (finsetSumComplEquiv I hI) (finsetSumComplEquiv J hJ)

end
```

---

*User prompt: ✖ [1796/1796] Building DetPoly.Scratch (946ms) trace: .> LEAN_PATH=/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/MD4Lean/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/BibtexQuery/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/UnicodeBasic/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/Cli/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/leansqlite/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/batteries/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/Qq/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/aesop/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/proofwidgets/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/importGraph/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/LeanSearchClient/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/plausible/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/checkdecls/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/doc-gen4/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/packages/mathlib/.lake/build/lib/lean:/home/grant/proj/lean/Determinantal-Polynomials/.lake/build/lib/lean /home/grant/.elan/toolchains/leanprover--lean4---v4.35.0/bin/lean /home/grant/proj/lean/Determinantal-Polynomials/DetPoly/Scratch.lean -o /home/grant/proj/lean/Determinantal-Polynomials/.lake/build/lib/lean/DetPoly/Scratch.olean -i /home/grant/proj/lean/Determinantal-Polynomials/.lake/build/lib/lean/DetPoly/Scratch.ilean -c /home/grant/proj/lean/Determinantal-Polynomials/.lake/build/ir/DetPoly/Scratch.c --setup /home/grant/proj/lean/Determinantal-Polynomials/.lake/build/ir/DetPoly/Scratch.setup.json --json error: DetPoly/Scratch.lean:103:2: Type mismatch: After simplification, term h has type (A.submatrix (⇑(Equiv.sumCompl fun x ↦ x ∈ I) ∘ ⇑((Equiv.refl ↥I).sumCongr { toFun := fun x ↦ ⟨↑x, ⋯⟩, invFun := fun x ↦ ⟨↑x, ⋯⟩, left_inv := ⋯, right_inv := ⋯ })) (⇑(Equiv.sumCompl fun x ↦ x ∈ I) ∘ ⇑((Equiv.refl ↥I).sumCongr { toFun := fun x ↦ ⟨↑x, ⋯⟩, invFun := fun x ↦ ⟨↑x, ⋯⟩, left_inv := ⋯, right_inv := ⋯ })).det * (A.adjugate.submatrix ((⇑(Equiv.sumCompl fun x ↦ x ∈ I) ∘ ⇑((Equiv.refl ↥I).sumCongr { toFun := fun x ↦ ⟨↑x, ⋯⟩, invFun := fun x ↦ ⟨↑x, ⋯⟩, left_inv := ⋯, right_inv := ⋯ })) ∘ Sum.inr) ((⇑(Equiv.sumCompl fun x ↦ x ∈ I) ∘ ⇑((Equiv.refl ↥I).sumCongr { toFun := fun x ↦ ⟨↑x, ⋯⟩, invFun := fun x ↦ ⟨↑x, ⋯⟩, left_inv := ⋯, right_inv := ⋯ })) ∘ Sum.inr)).det = A.det ^ (Fintype.card n - I.card) * (A.submatrix ((⇑(Equiv.sumCompl fun x ↦ x ∈ I) ∘ ⇑((Equiv.refl ↥I).sumCongr { toFun := fun x ↦ ⟨↑x, ⋯⟩, invFun := fun x ↦ ⟨↑x, ⋯⟩, left_inv := ⋯, right_inv := ⋯ })) ∘ Sum.inl) ((⇑(Equiv.sumCompl fun x ↦ x ∈ I) ∘ ⇑((Equiv.refl ↥I).sumCongr { toFun := fun x ↦ ⟨↑x, ⋯⟩, invFun := fun x ↦ ⟨↑x, ⋯⟩, left_inv := ⋯, right_inv := ⋯ })) ∘ Sum.inl)).det but is expected to have type A.det * (A.adjugate.submatrix Subtype.val Subtype.val).det = A.det ^ (Fintype.card n - k) * (A.submatrix Subtype.val Subtype.val).det Some required targets logged failures: - DetPoly.Scratch*

Response: The error on line 103 occurs because `simpa` simplifies inside-out: having `eI` in the simp set causes `simp` to unfold `⇑eI` and apply `@[simp] lemma Equiv.coe_trans (⇑(e₁.trans e₂) = ⇑e₂ ∘ ⇑e₁)` *before* reaching the outer `(A.submatrix ⇑eI ⇑eI).det`, which prevents `det_submatrix_equiv_self` from matching.

Rewriting with `det_submatrix_equiv_self` and `hcard` before unfolding `eI` fixes the proof, since `eI ∘ Sum.inl` and `eI ∘ Sum.inr` are definitionally equal to `Subtype.val`.

Here is the complete, updated file:

```
module

public import Mathlib.LinearAlgebra.Matrix.Adjugate
public import Mathlib.LinearAlgebra.Matrix.Block
public import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
public import Mathlib.Data.Finset.Sort

/-!
# Jacobi's Complementary Minor Identity

This file proves that for an `n × n` matrix `A` over a commutative ring `R`,
`det(A) * A^{\text{adj}}[J^c, I^c] = \text{sgn}(\sigma_{I,J}) • (det(A)^{n-k} * A[I, J])`.
-/

public section

open Matrix
```

```
variable {n m ι₁ ι₂ R : Type*}
  [Fintype n] [DecidableEq n]
  [Fintype m] [DecidableEq m]
  [Fintype ι₁] [DecidableEq ι₁]
  [Fintype ι₂] [DecidableEq ι₂]
  [CommRing R]

/-- Reindexing rows by `eI` and columns by `eJ` multiplies the determinant by
the sign of the relative permutation `eJ.symm.trans eI`. -/
lemma det_submatrix_equiv (A : Matrix n n R) (eI eJ : m ≃ n) :
    (A.submatrix eI eJ).det = Equiv.Perm.sign (eJ.symm.trans eI) • A.det := by
  have h : A.submatrix eI eJ = (A.submatrix (eJ.symm.trans eI) id).submatrix eJ eJ := by
    ext i j
    simp
  rw [h, det_submatrix_equiv_self, det_permute, Units.smul_def, zsmul_eq_mul]

/-- **Core Block-Matrix Identity**:
For any row partition `eI : ι₁ ⊕ ι₂ ≃ n` and column partition `eJ : ι₁ ⊕ ι₂ ≃ n`,
`(A.submatrix eI eJ).det * A^{\text{adj}}[J^c, I^c] = A.det ^ |ι₂| * A[I, J]`. -/
theorem det_submatrix_mul_adjugate_minor (A : Matrix n n R) (eI eJ : ι₁ ⊕ ι₂ ≃ n) :
    (A.submatrix eI eJ).det *
      (A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)).det =
    A.det ^ Fintype.card ι₂ *
      (A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)).det := by
  let A₁₁ := A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)
  let A₁₂ := A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inr)
  let A₂₁ := A.submatrix (eI ∘ Sum.inr) (eJ ∘ Sum.inl)
  let A₂₂ := A.submatrix (eI ∘ Sum.inr) (eJ ∘ Sum.inr)
  let B₁₁ := A.adjugate.submatrix (eJ ∘ Sum.inl) (eI ∘ Sum.inl)
  let B₁₂ := A.adjugate.submatrix (eJ ∘ Sum.inl) (eI ∘ Sum.inr)
  let B₂₁ := A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inl)
  let B₂₂ := A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)
  have hA : A.submatrix eI eJ = fromBlocks A₁₁ A₁₂ A₂₁ A₂₂ := by
    ext (i | i) (j | j) <;> rfl
  have hB : A.adjugate.submatrix eJ eI = fromBlocks B₁₁ B₁₂ B₂₁ B₂₂ := by
    ext (i | i) (j | j) <;> rfl
  -- Expand `(A * A.adjugate).submatrix eI eI = A.det • 1` in block form
  have hAB : fromBlocks A₁₁ A₁₂ A₂₁ A₂₂ * fromBlocks B₁₁ B₁₂ B₂₁ B₂₂ =
      fromBlocks (A.det • 1) 0 0 (A.det • 1) := by
    rw [← hA, ← hB, submatrix_mul_equiv, mul_adjugate]
    ext (i | i) (j | j) <;> simp [one_apply]
  rw [fromBlocks_multiply, fromBlocks_inj] at hAB
  obtain ⟨-, h₁₂, -, h₂₂⟩ := hAB
  -- Multiply the reindexed `A` by the block upper-triangular matrix `fromBlocks 1 B₁₂ 0 B₂₂`
  have h_mul : A.submatrix eI eJ * fromBlocks 1 B₁₂ 0 B₂₂ =
      fromBlocks A₁₁ 0 A₂₁ (A.det • 1) := by
    rw [hA, fromBlocks_multiply]
    simp [h₁₂, h₂₂]
  calc (A.submatrix eI eJ).det * B₂₂.det
    _ = (A.submatrix eI eJ * fromBlocks 1 B₁₂ 0 B₂₂).det := by
        rw [det_mul, det_fromBlocks_zero₂₁, det_one, one_mul]
```

```
    _ = (fromBlocks A₁₁ 0 A₂₁ (A.det • 1)).det := by
        rw [h_mul]
    _ = A.det ^ Fintype.card ι₂ * A₁₁.det := by
        rw [det_fromBlocks_zero₁₂, det_smul, det_one, mul_one, mul_comm]

/-- **Jacobi's Complementary Minor Identity (General Partition Form)**:
`det(A) * A^{\text{adj}}[J^c, I^c] = sgn(eJ⁻¹ ∘ eI) • (det(A)^{|I^c|} * A[I, J])`. -/
theorem det_mul_adjugate_minor (A : Matrix n n R) (eI eJ : ι₁ ⊕ ι₂ ≃ n) :
    A.det * (A.adjugate.submatrix (eJ ∘ Sum.inr) (eI ∘ Sum.inr)).det =
    Equiv.Perm.sign (eJ.symm.trans eI) •
      (A.det ^ Fintype.card ι₂ * (A.submatrix (eI ∘ Sum.inl) (eJ ∘ Sum.inl)).det) := by
  have h := det_submatrix_mul_adjugate_minor A eI eJ
  rw [det_submatrix_equiv, smul_mul_assoc] at h
  rw [← h, smul_smul, Int.units_mul_self, one_smul]

/-- **Unsigned Principal Minor Identity (`I = J`)**:
For any subset `I : Finset n` of cardinality `k`,
`det(A) * A^{\text{adj}}[I^c, I^c] = det(A)^{n - k} * A[I, I]`. -/
theorem det_mul_adjugate_principal_minor (A : Matrix n n R) (I : Finset n)
    {k : ℕ} (hI : I.card = k) :
    A.det * (A.adjugate.submatrix (Subtype.val : ↥(Iᶜ) → n) (Subtype.val : ↥(Iᶜ) → n)).det =
    A.det ^ (Fintype.card n - k) *
      (A.submatrix (Subtype.val : ↥I → n) (Subtype.val : ↥I → n)).det := by
  let eCompl : {x // x ∉ I} ≃ ↥(Iᶜ) :=
    { toFun := fun x => ⟨x.1, Finset.mem_compl.mpr x.2⟩
      invFun := fun x => ⟨x.1, Finset.mem_compl.mp x.2⟩
      left_inv := fun _ => rfl
      right_inv := fun _ => rfl }
  let eI : ↥I ⊕ ↥(Iᶜ) ≃ n :=
    (Equiv.sumCongr (Equiv.refl ↥I) eCompl.symm).trans (Equiv.sumCompl (· ∈ I))
  have h := det_submatrix_mul_adjugate_minor A eI eI
  have hcard : Fintype.card ↥(Iᶜ) = Fintype.card n - k := by
    rw [Fintype.card_coe, Finset.card_compl, hI]
  rw [det_submatrix_equiv_self, hcard] at h
  exact h

/-- Canonical equivalence `Fin k ⊕ Fin (n - k) ≃ Fin n` induced by `I : Finset (Fin n)`
of cardinality `k`, preserving the natural order on `I` and `Iᶜ`. -/
noncomputable def finsetSumComplEquiv {n k : ℕ} (I : Finset (Fin n)) (hI : I.card = k) :
    Fin k ⊕ Fin (n - k) ≃ Fin n :=
  have hIc : (Iᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hI]
  let e₁ : Fin k ≃ ↥I := (I.orderIsoOfFin hI).toEquiv
  let e₂ : Fin (n - k) ≃ {x // x ∉ I} :=
    (Iᶜ.orderIsoOfFin hIc).toEquiv.trans
      { toFun := fun x => ⟨x.1, Finset.mem_compl.mp x.2⟩
        invFun := fun x => ⟨x.1, Finset.mem_compl.mpr x.2⟩
        left_inv := fun _ => rfl
        right_inv := fun _ => rfl }
  (Equiv.sumCongr e₁ e₂).trans (Equiv.sumCompl (· ∈ I))

/-- **Jacobi's Complementary Minor Identity for `I, J ⊆ [n]`**:
Given `I, J : Finset (Fin n)` of size `k`, denotes `A[I, J]` and `A^{\text{adj}}[J^c, I^c]`
```

```
via their canonical order-preserving enumerations `orderIsoOfFin`. -/
theorem det_mul_adjugate_finset_minor {n k : ℕ} (A : Matrix (Fin n) (Fin n) R)
    (I J : Finset (Fin n)) (hI : I.card = k) (hJ : J.card = k)
    (hIc : (Iᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hI])
    (hJc : (Jᶜ).card = n - k := by rw [Finset.card_compl, Fintype.card_fin, hJ]) :
    A.det *
      (A.adjugate.submatrix
        (Subtype.val ∘ (Jᶜ.orderIsoOfFin hJc))
        (Subtype.val ∘ (Iᶜ.orderIsoOfFin hIc))).det =
    Equiv.Perm.sign ((finsetSumComplEquiv J hJ).symm.trans (finsetSumComplEquiv I hI)) •
      (A.det ^ (n - k) *
        (A.submatrix
          (Subtype.val ∘ (I.orderIsoOfFin hI))
          (Subtype.val ∘ (J.orderIsoOfFin hJ))).det) := by
  simpa [finsetSumComplEquiv, Function.comp_def] using
    det_mul_adjugate_minor A (finsetSumComplEquiv I hI) (finsetSumComplEquiv J hJ)

end
```